

Exercice d'application NumPy

Le POKER :

Cet exercice est en lien avec ma chaîne YouTube : **Mycode-Developpement**

Les probabilités au poker te semblent un mystère ? Grâce à **NumPy**, tu vas les calculer toi-même et voir si la chance est vraiment de ton côté !

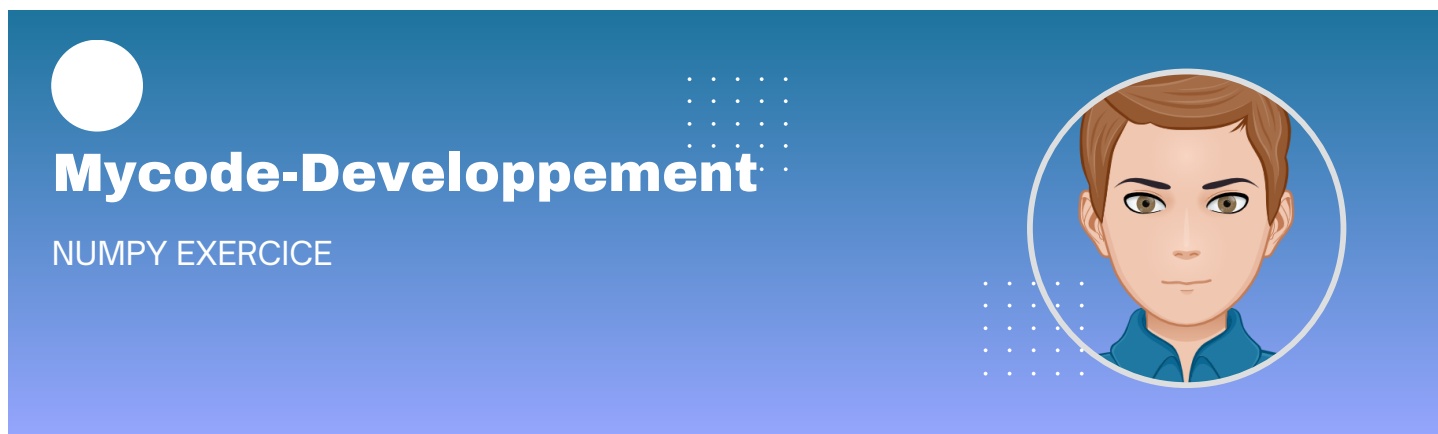
Dans cet exercice, nous allons calculer les probabilités d'obtenir différentes combinaisons au poker.

Remarque : Nous ne prendrons pas en compte les couleurs des cartes afin de simplifier certaines parties du code.

	<i>Combinaison</i>	<i>Probabilité %</i>
<i>Carré</i>	624	0.024
<i>Full</i>	3744	0.144
<i>Brelan</i>	54912	2.113
<i>Doubles Paires</i>	123552	4.754
<i>Paires</i>	1098240	42.257

Figure 1  : Les probabilités des différentes mains

Pour obtenir une **estimation proche des probabilités réelles**, nous allons simuler **des milliers de parties** afin d'observer la **convergence vers ces valeurs théoriques**.





Question à suivre

Pour plus d'aide, voir la section aide supplémentaire : les fonctions sont données

1- Installer et importer **NumPy** pour la manipulation des tableaux.

2- Définir un tableau `jeu_initial` contenant les cartes de **1 à 13** (11 = Valet, 12 = Dame, 13 = Roi), avec **4 occurrences de chaque valeur** (une par couleur, même si elles ne sont pas différenciées).



Aides

`jeu_initial = [ 1, 2, ..., 13, 1, 2, ..., 13, 1, 2, ..., 13, 1, 2, ..., 13 ]`

3- **Mélanger** le jeu de carte à l'aide de fonction Numpy

4- Créer une variable **main** contenant 5 cartes du jeu initial, vous pouvez **l'afficher** dans la console et vous assurez que ce résultat est bien **aléatoire**.

5- Créer un **tableau comptant le nombre d'apparitions de chaque valeur** dans la main comme sur l'exemple ci-joint

```
#Exemple
main = [11  4  9  3 12]
output = [1 1 1 1 1] #-> résultat attendu
#Exemple 2
main = [ 4 11  7  9  9]
output = [1 1 1 2] #-> résultat attendu
```

6- Pour comparer les tableaux et les associer aux différentes mains, nous allons leur attribuer un **code unique** en effectuant des calculs sur les tableaux. Cela permettra de les convertir en valeurs réelles, évitant ainsi les erreurs liées à la comparaison directe de tableaux NumPy de tailles variables.

7- Définir les **variables principales**, notamment **N** pour représenter le nombre de répétitions, ainsi qu'une variable pour chaque type de main (exemple : **nbr_paires** = 0).

8- Créer une **boucle principale** qui répétera **tant que le nombre de carré est inférieur à 15**, la génération d'une main, en incrémentant les variables comptant les occurrences de chaque type de combinaison (paire, double paire, etc.) à chaque apparition.

9- **Afficher les probabilités** dans la console et les comparer avec le tableau des probabilités théoriques présenté précédemment.



Résultat attendu :

P_nb_carre 0.024965879964049132

Pnb_full 0.14646649578908824

Pnb_brelan 2.1487300689058286

Pnb_doublepaire 4.785126993109417

Pnb_paire 42.12909024333411



Aides

Petites aides supplémentaires

Pour toutes les fonctions, vous pouvez regarder la documentation sur internet !

- 1- Utiliser la commande dans le terminal si le module n'est pas installé : **pip install numpy**
- 2- Fonction **np.arange** et **np.concatenate** peuvent être utiles
- 3- **np.random.permutation()**
- 4- Rien à ajouter
- 5- **np.bincount()** et **np.trim_zeros()** et **np.sort()**
- 6- Pour créer notre code unique on peut **élever** chaque chiffre au carré et faire la **somme** du tableau. Les possibilités de tableau sont : *output = [1,4] ou [2,3] ou [3,1,1] ou [2,2,1] ou [2,1,1,1] ou [1,1,1,1,1]*
- 7- Rien à ajouter
- 8- Une boucle **while** serait le plus judicieux
- 9- Rien à ajouter

N'hésite pas à aller voir la vidéo YouTube pour une meilleur explication !

